Análisis de Algoritmos

Dra. Irene Olaya Ayaquica Martínez
Facultad de Ciencias de la Computación, BUAP
irene.ayaquica@correo.buap.mx

Enfoques del análisis de algoritmos

- **Enfoque empírico (a posteriori):**
  - Programar las técnicas competidoras e irlas probando en distintos casos con ayuda de una computadora.

- **Enfoque teórico (a priori):**
  - Determinar matemáticamente la cantidad de recursos necesarios para cada uno de los algoritmos como función del tamaño de los casos considerados.
  - Tiempo de ejecución y espacio de almacenamiento.
  - Eficiencia -> rapidez de ejecución

Enfoques del análisis de algoritmos

- **Enfoque hibrido:**
  - La forma de la función que describe la eficiencia del algoritmo se determina teóricamente, y empíricamente se determinan aquellos parámetros numéricos que sean específicos para cierto programa y para cierta máquina.

  - Se puede predecir el tiempo que necesitará una cierta implementación para resolver un ejemplar mucho mayor que los que se hayan empleado en las pruebas.

Enfoques del análisis de algoritmos

- Si deseamos medir la cantidad de espacio que utiliza un algoritmo en función del tamaño de los ejemplares, se tiene el bit.
  - Esta unidad de medida no depende de la computadora.

- Si deseamos medir la eficiencia de un algoritmo en términos del tiempo no contamos con una medida tan evidente.

Principio de invariancia

- Dos implementaciones distintas de un mismo algoritmo no diferirán en su eficiencia en más de alguna constante multiplicativa.

- Si dos implementaciones del mismo algoritmo necesitan $t_1(n)$ y $t_2(n)$ segundos respectivamente, para resolver un caso de tamaño n, entonces siempre existen constantes positivas c y d tales que $t_1(n) \leq ct_2(n)$ y $t_2(n) \leq dt_1(n)$. Siempre que n sea suficientemente grande.

Principio de invariancia

- Diremos que un algoritmo para algún problema requiere un tiempo del orden $t(n)$ para una función dada t, si existen una constante positiva c y una implementación del algoritmo capaz de resolver todos los casos de tamaño n en un tiempo que no sea superior a $ct(n)$ segundos.

Nombres de los órdenes

- Si el tiempo necesario para que un algoritmo resuelva un caso del tamaño n nunca es más que cn segundos, en donde c es alguna constante adecuada, diremos que el algoritmo requiere un tiempo en el orden de n o que requiere un **tiempo lineal**. (Algoritmo lineal).

- Si el algoritmo nunca necesita más de cn^2 segundos, entonces el algoritmo requiere un tiempo en el orden de n^2 o **tiempo cuadrático.** (Algoritmo cuadrático).

- $n^3 \rightarrow$ tiempo cúbico, $n^k \rightarrow$ tiempo polinómico, $c^n \rightarrow$ tiempo exponencial.

Operaciones elementales (OE)

- Una operación elemental es aquella cuyo tiempo de ejecución se puede acotar superiormente por una constante que solamente dependerá de la implementación particular:
  - de la máquina,
  - del lenguaje de programación,
  - etc.

- La constante no depende ni del tamaño ni de los parámetros del ejemplar que se está considerando.

Operaciones elementales (OE)

- Consideraremos operaciones elementales:
  - Operaciones aritméticas básicas,
  - Asignaciones a variables de tipo predefinido por el compilador,
  - Saltos (llamadas a funciones, retorno, etc.),
  - Comparaciones lógicas,
  - Acceso a estructuras indexadas básicas (vectores, matrices, etc.)

- Cada una de ellas contabilizará como una operación elemental.

Operaciones elementales (OE)

- El tiempo de ejecución de un algoritmo va a ser una función que mide el número de operaciones elementales que realiza el algoritmo para un tamaño de entrada dado.

Eficiencia y complejidad

- Supongamos que cuando se analiza un algoritmo, encontramos que para resolver un caso de un cierto tamaño se necesita efectuar a adiciones, m multiplicaciones y s asignaciones.

- Supongamos que también se sabe que una suma nunca requiere más de t_a microsegundos, que una multiplicación nunca requiere más de t_m microsegundos y una asignación nunca requiere más de t_s microsegundos; donde t_a, t_m y t_s son constantes que dependen de la máquina utilizada.

Eficiencia y complejidad

- El tiempo total t requerido por nuestro algoritmo estará acotado por:

$$t \leq at_a + mt_m + st_s \leq \max(t_a, t_m, t_s) * (a + m + s)$$

- esto es, t está acotado por un múltiplo constante del número de operaciones elementales que hay que ejecutar.

Eficiencia y complejidad

- Si T es una matriz de n elementos ($n > 0$), entonces el tiempo requerido para calcular:

$$x \leftarrow min\{T[i] \mid 1 \leq i \leq n\}$$

crece con n, puesto que esto es una abreviatura de:

$$x \leftarrow T[1]$$
$$para\ i \leftarrow 2\ hasta\ n\ hacer$$
$$\quad si\ T[i] < x\ entonces\ x \leftarrow T[i]$$

Eficiencia y complejidad

- $función\ Sum(n)$

$sum \leftarrow 0$	1
$\boldsymbol{para\ i \leftarrow 1\ hasta\ n\ hacer}$	$\boldsymbol{4}$
$\quad sum \leftarrow sum + i$	2
$devolver\ sum$	1

- Calcula la suma de los enteros de 1 a n.

Eficiencia y complejidad

- $función\ Fibonacci(n)$

$i \leftarrow 1$	1
$j \leftarrow 0$	1
$\boldsymbol{para\ k \leftarrow 1\ hasta\ n\ hacer}$	$\boldsymbol{4}$
$\quad j \leftarrow i + 1$	2
$\quad i \leftarrow j - 1$	2
$devolver\ j$	1

- Calcula el n-ésimo término de la sucesión de Fibonacci.

Reglas para el cálculo del número de OE

- El tiempo de una operación elemental (OE) es de orden 1.
- El tiempo de ejecución de una secuencia consecutiva de instrucciones se calcula sumando los tiempos de ejecución de cada una de las instrucciones.
- El tiempo de ejecución de la sentencia "$Case\ C\ of\ v_1 : S_1 \mid v_2 : S_2 \mid \ldots \mid v_n : S_n$" es $T = T(C) + \max\{T(S_1), T(S_2), \ldots, T(S_n)\}$. $T(C)$ incluye el tiempo de comparación con $v_1, v_2, \ldots, v_n$.

Reglas para el cálculo del número de OE

- El tiempo de ejecución de la sentencia "$If\ C\ then\ S_1\ else\ S_2$" es $T = T(C) + \max\{T(S_1), T(S_2)\}$.

- El tiempo de ejecución de un bucle de sentencias "$While\ C\ do\ S$" es $T = T(C) + (no.iteraciones) * (T(S) + T(C))$. Tanto $T(S)$ como $T(C)$ pueden variar en cada iteración.

- Para calcular el tiempo de ejecución del resto de sentencias iterativas $(For, Repeat, Loop)$ basta expresarlas como un bucle $While$.

Reglas para el cálculo del número de OE

$For\ i \coloneqq 1\ to\ n\ do$
    S
$End;$

$i \coloneqq 1$
$While\ i \leq n\ do$
    S
    $i \coloneqq i + 1$
$End;$

Reglas para el cálculo del número de OE

- El tiempo de ejecución de una llamada a un procedimiento o función $F(P_1, P_2, \ldots, P_n)$ es 1 (por la llamada) más el tiempo de evaluación de los parámetros $P_1, P_2, \ldots, P_n$ más el tiempo que tarde en ejecutarse F.

$$T = 1 + T(P_1) + T(P_2) + \cdots + T(P_n) + T(F)$$

- El tiempo de ejecución de las llamadas a procedimientos recursivos da lugar a ecuaciones de recurrencia.

Eficiencia y complejidad

- $función\ Sum(n)$

$sum \leftarrow 0$	1
$\boldsymbol{i \leftarrow 1}$	$\boldsymbol{1}$
$\boldsymbol{mientras\ i \leq\ n\ hacer}$	$\boldsymbol{1}$
$\quad sum \leftarrow sum + i$	2
$\quad \boldsymbol{i \leftarrow i + 1}$	$\boldsymbol{2}$
$devolver\ sum$	1

- Calcula la suma de los enteros de 1 a n.

Eficiencia y complejidad

- $función\ Fibonacci(n)$

$i \leftarrow 1$	1
$j \leftarrow 0$	1
$\boldsymbol{k \leftarrow 1}$	**1**
$\boldsymbol{mientras\ k \leq\ n\ hacer}$	**1**
$\quad j \leftarrow i + 1$	2
$\quad i \leftarrow j - 1$	2
$\quad \boldsymbol{k \leftarrow k + 1}$	**2**
$devolver\ j$	1

- Calcula el n-ésimo término de la sucesión de Fibonacci.

Eficiencia y complejidad

- También es importante hacer notar que el comportamiento de un algoritmo puede cambiar notablemente para diferentes entradas (por ejemplo, lo ordenados que se encuentren ya los datos a ordenar).

- Para muchos programas el tiempo de ejecución es una función de la entrada específica, y no sólo del tamaño de ésta.

Eficiencia y complejidad

- Suelen estudiarse tres casos para un mismo algoritmo: *caso peor, caso mejor y caso medio*.

- El **caso mejor** corresponde a la traza (secuencia de sentencias) del algoritmo que realiza menos instrucciones.

- El **caso peor** corresponde a la traza del algoritmo que realiza más instrucciones.

Eficiencia y complejidad

- El **caso medio**, corresponde a la traza del algoritmo que realiza un número de instrucciones igual a la esperanza matemática de la variable aleatoria definida por todas las posibles trazas del algoritmo para un tamaño de la entrada dado, con las probabilidades de que éstas ocurran para esa entrada.

Ejemplo

```
Procedure Buscar(var a: vector; c: integer)
    var j
    Begin
        j := 1;                                  (1)  1
        while (a[j] < c)and(j < n) do            (2)  4
            j := j + 1                           (3)  2
        end;                                     (4)
        if a[j] = c then                         (5)  2
            return j                             (6)  1
        else                                     (7)
            return 0                             (8)  1
        end                                      (9)
25  end Buscar
```

Ejemplo

- Para determinar el tiempo de ejecución, calculamos primero el número de operaciones elementales que se realizan:
  - En la línea (1) se ejecuta 1 OE (asignación)
  - En la línea (2) se efectúa la condición del bucle, con un total de 4 OE (2 comparaciones, un acceso a vector y un and)
  - La línea (3) realiza 2 OE (1 incremento y 1 asignación)
  - En la línea (5) se realizan 2 OE (1 condición y 1 acceso al vector)
  - La línea (6) contiene 1 OE (return, la cond. se cumple)
  - La línea (8) contiene 1 OE (return, la cond. no se cumple)

Ejemplo

- En el **caso mejor**, el elemento está en la primera posición.
- Se ejecutará la línea (1)
- De la línea (2) sólo la primera mitad de la condición (2 OE). (suponemos que las expresiones se evalúan de izquierda a derecha, y con "cortocircuito", es decir, una expresión lógica deja de ser evaluada en el momento que se conoce su valor, aunque no hayan sido evaluados todos sus términos).

Ejemplo

- Después ejecuta las líneas (5) y (6).
- Por lo tanto,

$$T(n) = 1 + 2 + 3$$
$$T(n) = 6$$

- **Tiempo constante**.

Ejemplo

- En el **caso peor**, el elemento no está en el vector.

- Se ejecutará la línea (1), el bucle se repite $n-1$ veces mientras se cumple la segunda condición, después se efectúa la condición de la línea (5) y la ejecución acaba con la línea (8).

- Cada iteración del bucle está compuesta por las líneas (2) y (3), junto con la ejecución adicional de la línea (2) que es la que ocasiona la salida del bucle.

Ejemplo

$$\sum_{i=1}^{n} c = cn$$

- Por lo tanto,

$$T(n) = 1 + \left[4 + \left(\sum_{i=1}^{n-1} (4+2)\right)\right] + 2 + 1$$

$$T(n) = 1 + [4 + 6(n-1)] + 3$$

$$T(n) = 8 + 6n - 6$$

$$T(n) = 6n + 2$$

- **Tiempo lineal**.

Ejemplo

- En el **caso medio**, el elemento está en cualquier posición del vector.

- El bucle se efectuará un número de veces entre 0 y $n-1$ y vamos a suponer que cada una de ellas tiene la misma probabilidad de suceder.

- Como existen n posibilidades (puede ser que el número buscado no esté) suponemos a priori que son equiprobables y por lo tanto cada una tendrá una probabilidad asociada de $1/n$.

Ejemplo

- Con esto, el número de veces que se efectuará el bucle es de:

$$\sum_{i=0}^{n-1} i\frac{1}{n} = \frac{n-1}{2}$$

- Por lo tanto,

$$T(n) = 1 + \left[2 + \left(\sum_{i=1}^{(n-1)/2} (4+2)\right)\right] + 2 + 1$$

$$T(n) = 3n + 3$$

- **Tiempo lineal**.

Ejemplo

- Es importante observar que no es necesario conocer el propósito del algoritmo para analizar su tiempo de ejecución y determinar sus casos mejor, peor y medio, sino que basta con estudiar su código.

- Suele ser un error muy frecuente el determinar tales casos basándose sólo en la funcionalidad para la que el algoritmo fue concebido, olvidando que es el código implementado el que los determina.

Ejemplo

- En este caso, un examen más detallado de la función (¡y no de su nombre!) nos muestra que tras su ejecución, la función devuelve la posición de un entero dado *c* dentro de un vector ordenado de enteros, devolviendo 0 si el elemento no está en el vector.

- Lo que acabamos de probar es que su **caso mejor** se da cuando el elemento está en la primera posición del vector.

Ejemplo

- El **caso peor** se produce cuando el elemento no está en el vector.

- El **caso medio** ocurre cuando consideramos equiprobables cada una de las posiciones en las que puede encontrarse el elemento dentro del vector (incluyendo la posición especial 0, que indica que el elemento a buscar no se encuentra en el vector).

Ejemplo

- $función\ Sum(n)$

$sum \leftarrow 0$	1
$i \leftarrow 1$	1
$mientras\ i \leq\ n\ hacer$	1
$\quad sum \leftarrow sum + i$	2
$\quad i \leftarrow i + 1$	2
$devolver\ sum$	1

- Calcula la suma de los enteros de 1 a n.

Ejemplo

- Se tiene un solo caso ya que el ciclo mientras proviene de un ciclo para, el cuál se ejecuta siempre.

$$T(n) = 1 + 1 + \left[1 + \left(\sum_{i=1}^{n}(1 + 2 + 2)\right)\right] + 1$$

$$T(n) = 4 + \left(\sum_{i=1}^{n} 5\right)$$

$$T(n) = 5n + 4$$

Tiempo lineal

Eficiencia y complejidad

- $función\ Fibonacci(n)$

$i \leftarrow 1$	1
$j \leftarrow 0$	1
$k \leftarrow 1$	1
$mientras\ k \leq\ n\ hacer$	1
$\quad j \leftarrow i + 1$	2
$\quad i \leftarrow j - 1$	2
$\quad k \leftarrow k + 1$	2
$devolver\ j$	1

- Calcula el n-ésimo término de la sucesión de Fibonacci.

Eficiencia y complejidad

- $T(n) = 1 + 1 + 1 + [1 + \sum_{k=1}^{n}(1 + 2 + 2 + 2)] + 1$
- $T(n) = 5 + \sum_{k=1}^{n} 7$
- $T(n) = 5 + 7n = 7n + 5$
- Tiempo lineal

Ejemplo

- Se tiene un solo caso ya que el ciclo mientras proviene de un ciclo para, el cuál se ejecuta siempre.

$$T(n) = 1 + 1 + 1 + \left[1 + \sum_{k=1}^{n} (1 + 2 + 2 + 2)\right] + 1$$

$$T(n) = 5 + \left[\sum_{k=1}^{n} (7)\right]$$

$$T(n) = 5 + 7n = 7n + 5$$

Tiempo lineal

Eficiencia y complejidad

- $Procedimiento\ Insertar(T[1, \dots, n])$

  $para\ i \leftarrow 2\ hasta\ n\ hacer$

  $x \leftarrow T[i]$

  $j \leftarrow i - 1$

  $mientras\ j > 0\ y\ x < T[j]\ hacer$

  $T[j + 1] \leftarrow T[j]$

  $j \leftarrow j - 1$

  $T[j + 1] \leftarrow x$

Eficiencia y complejidad

- $Procedimiento\ Insertar(T[1, \dots, n])$

$i \leftarrow 2$	*1*
$mientras\ i \leq n\ hacer$	*1*
$x \leftarrow T[i]$	*2*
$j \leftarrow i - 1$	*2*
$mientras\ j > 0\ y\ x < T[j]\ hacer$	*4*
$T[j + 1] \leftarrow T[j]$	*4*
$j \leftarrow j - 1$	*2*
$T[j + 1] \leftarrow x$	*3*
$i \leftarrow i + 1$	*2*

Ejemplo

$$n = 5 \qquad k = 1$$

$$T = [4\ 1\ 3\ 5\ 2]$$

$$i \leftarrow 2$$

$$x \leftarrow T[2] = 1$$

$$j \leftarrow 1$$

$$T[2] \leftarrow T[1] \quad T[2] \leftarrow 4$$

$$j \leftarrow 0$$

$$T[1] \leftarrow 1$$

El mientras se ejecutó 1 vez

Ejemplo

$$n = 5 \qquad k = 2$$

$$T = [1\ 4\ 3\ 5\ 2]$$

$$i \leftarrow 3$$

$$x \leftarrow T[3] = 3$$

$$j \leftarrow 2$$

$$T[3] \leftarrow T[2] \quad T[3] \leftarrow 4$$

$$j \leftarrow 1$$

$$T[2] \leftarrow 3$$

El mientras se ejecutó 1 vez

Ejemplo

$$n = 5 \qquad k = 4$$

$$T = [1\ 3\ 4\ 5\ 2]$$

$$i \leftarrow 4$$

$$x \leftarrow T[4] = 5$$

$$j \leftarrow 3$$

$$T[4] \leftarrow 5$$

El mientras se ejecutó 0 veces

Ejemplo

$$n = 5 \qquad k = 2$$

$$T = [1\ 3\ 4\ 5\ 2]$$

$$i \leftarrow 5$$

$$x \leftarrow T[5] = 2$$

$$j \leftarrow 4$$

$$T[5] \leftarrow T[4] \quad T[5] \leftarrow 5$$

$$j \leftarrow 3$$

$$T[4] \leftarrow T[3] \quad T[4] \leftarrow 4$$

$$j \leftarrow 2$$

$$T[3] \leftarrow T[2] \quad T[3] \leftarrow 3$$

$$j \leftarrow 1$$

$$T[2] \leftarrow 2$$

El mientras se ejecutó 3 veces

Ejemplo

$$n = 5$$
$$T = [1\ 2\ 3\ 4\ 5]$$
$$i \leftarrow 6$$

Ejemplo

Caso mejor: vector ordenado (menor a mayor)

$$T(n) = 1 + \left[1 + \sum_{i=2}^{n} (1 + 2 + 2 + [4] + 3 + 2)\right]$$

$$T(n) = 2 + \sum_{i=2}^{n} 14 = 2 + \sum_{i=1}^{n-1} 14$$

$$T(n) = 2 + 14n - 14 = \mathbf{14n} - 12$$

- **Tiempo lineal.**

Ejemplo

Caso peor: vector ordenado al revés (mayor a menor)

$$T(n)$$

$$= 1 + \left[1 + \sum_{i=2}^{n} \left(1 + 2 + 2 + \left[4 + \sum_{j=1}^{i-1} (4 + 4 + 2)\right] + 3 + 2 \right)\right]$$

$$T(n) = 2 + \sum_{i=2}^{n} \left(14 + \sum_{j=1}^{i-1} 10 \right)$$

Ejemplo

$$\sum_{i=1}^{n} ci = c\left(\frac{n(n+1)}{2}\right)$$

$$T(n) = 2 + \sum_{i=2}^{n} (14 + 10(i-1))$$

$$T(n) = 2 + \sum_{i=2}^{n} (14 + 10i - 10)$$

$$T(n) = 2 + \sum_{i=1}^{n-1} 14 + \sum_{i=1}^{n-1} 10i - \sum_{i=1}^{n-1} 10$$

Ejemplo

$$T(n) = 2 + 14(n-1) + 10\left(\frac{(n-1)n}{2}\right) - 10(n-1)$$

$$T(n) = 2 + 14n - 14 + 10\left(\frac{n^2 - n}{2}\right) - 10n + 10$$

$$T(n) = 2 + 14n - 14 + 5n^2 - 5n - 10n + 10$$

$$T(n) = 5n^2 - n - 2$$

- **Tiempo cuadrático.**

Ejemplo

Caso medio: vector desordenado

- Para cada valor de i, la probabilidad de que el elemento se sitúe en alguna posición k de las i primeras será de $^1/_i$.
- El número de veces que se repite el mientras interno es $(i - k)$.
- Entonces:

Ejemplo

$$\left[\frac{1}{i}\sum_{k=1}^{i} 10(i-k)\right] + 4$$

$$= \left[\frac{10}{i}\sum_{k=1}^{i} (i-k)\right] + 4$$

$$= \left[\frac{10}{i}\sum_{k=1}^{i} i - \frac{10}{i}\sum_{k=1}^{i} k\right] + 4$$

Ejemplo

$$= \left[\frac{10i^2}{i} - \frac{10}{i}\left(\frac{i(i+1)}{2}\right)\right] + 4$$

$$= \left[\frac{10i^2}{i} - \frac{10i^2 + 10i}{2i}\right] + 4$$

$$= \left[\frac{20i^2 - 10i^2 - 10i}{2i}\right] + 4$$

$$= [10i - 5i - 5] + 4 = 5i - 1$$

Ejemplo

Caso medio: vector desordenado

$$T(n) = 1 + \left[1 + \sum_{i=2}^{n} (1 + 2 + 2 + [4 + (5i - 1)] + 3 + 2)\right]$$

$$T(n) = 2 + \sum_{i=2}^{n} \big(14 + (5i - 1)\big) = 2 + \sum_{i=2}^{n} (13 + 5i)$$

$$T(n) = \ 2 + \sum_{i=1}^{n-1} 13 + \sum_{i=1}^{n-1} 5i = 2 + 13n - 13 + 5\left(\frac{(n-1)n}{2}\right)$$

Ejemplo

$$T(n) = 2 + 13n - 13 + 5\left(\frac{(n-1)n}{2}\right)$$

$$T(n) = 13n - 11 + \left(\frac{5n^2 - 5n}{2}\right)$$

$$T(n) = \frac{26n - 22 + 5n^2 - 5n}{2} = \frac{5}{2}n^2 + \frac{21}{2}n - 11$$

$$T(n) \in \Theta(n^2)$$

Tiempo cuadrático

Ejemplo

- $Procedure\ SumaDigitos(num)$

$VAR\ s;$

$BEGIN$

$s := num\ MOD\ 10;$ (1)

$WHILE\ num \geq 10\ \ DO$ (2)

$num := num\ DIV\ 10;$ (3)

$s := s + (num\ MOD\ 10);$ (4)

$RETURN\ s;$ (5)

$END\ SumaDigitos$

Ejemplo

- Para determinar el tiempo de ejecución, calculamos primero el número de operaciones elementales que se realizan:
  - En la línea (1) se ejecutan 2 OE (1 resto y 1 asignación)
  - En la línea (2) se efectúa la condición del bucle, con 1 OE
  - La línea (3) realiza 2 OE (1 asignación y 1 división)
  - En la línea (4) se realizan 3 OE (1 asignación, 1 suma y 1 resto)
  - La línea (5) contiene 1 OE

Ejemplo

- Se efectúa la línea (1), y después se repite el bucle hasta que su condición sea falsa, acabando la función al ejecutarse la línea (5).

- Cada iteración del bucle está compuesta por las líneas (2) a (4), junto con una ejecución adicional de la línea (2) que es la que ocasiona la salida del bucle.

- En cada iteración se diezman los elementos a considerar, por lo que el bucle se repite $log_{10}n$ veces.

Ejemplo

$$T(n) = 2 + \left[1 + \sum_{i=1}^{log_{10}n} (1 + 2 + 3)\right] + 1$$

$$T(n) = 6log_{10}n + 4$$

$$T(n) \in \Theta(logn)$$